

Résumé du projet

Ce notebook d'EDA explore le dataset des ventes aux USA d'Acme Co. pour la période 2014–2018 à travers :

Profilage et nettoyage des données : Vérification du schéma, traitement des budgets manquants et correction des types de données.

Analyse univariée et bivariable : Exploration des distributions (revenu, marge, prix unitaire), répartition par produit, canal et région, et segmentation des clients.

Tendances et saisonnalité : Visualisation des ventes mensuelles et annuelles, mettant en évidence les pics et creux récurrents.

Détection des valeurs aberrantes : Identification des transactions extrêmes aux deux extrémités des spectres de revenu et de prix unitaire.

Corrélation et segmentation : Évaluation des relations entre les métriques clés et clustering des clients selon le revenu et la marge bénéficiaire.

Problématique

Analyser les données de ventes d'Acme Co. (2014–2018) pour :

Identifier les principaux facteurs de revenu et de profit par produit, canal et région

Détecter les tendances saisonnières et les valeurs aberrantes

Comparer la performance par rapport aux budgets

Ces informations permettront d'optimiser la tarification, les promotions et l'expansion du marché pour une croissance durable et une réduction du risque de concentration.

Objectif

Fournir des insights exploitables à partir des ventes d'Acme Co. (2014–2018) afin de :

Identifier les produits, canaux et régions les plus performants en termes de revenu et de profit

Déceler les tendances saisonnières et anomalies pour un planning optimisé

Repérer les risques liés aux prix et aux marges via les transactions aberrantes

Orienter les stratégies de tarification, promotion et expansion du marché**

Setup & Configuration

In [2]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import warnings
warnings.filterwarnings("ignore")
```

In [3]:

```
sheets = pd.read_excel('Regional Sales Dataset.xlsx' , sheet_name = None)
```

In [4]:

```
# Assign to named DataFrames
df_sales      = sheets['Sales Orders']
df_customers  = sheets['Customers']
df_products   = sheets['Products']
df_regions    = sheets['Regions']
df_state_reg  = sheets['State Regions']
df_budgets    = sheets['2017 Budgets']
```

Data Profiling

In [5]:

```
for name, df in sheets.items():
    print(name, df.shape)
```

```
Sales Orders (64104, 12)
Customers (175, 2)
Regions (994, 15)
State Regions (49, 3)
Products (30, 2)
2017 Budgets (30, 2)
```

In [6]:

```
#Print first few rows for each sheet

print("\n- df_sales head -")
display(df_sales.head())
print()

print("\n- df_customers head -")
display(df_customers.head())
print()

print("\n- df_products head -")
display(df_products.head())
print()

print("\n- df_regions head -")
display(df_regions.head())
print()

print("\n- df_state_reg head -")
display(df_state_reg.head())
print()

print("\n- df_budgets head -")
display(df_budgets.head())
```

```
- df_sales head -
```

	OrderNumber	OrderDate	Customer Name Index	Channel	Currency Code	Warehouse Code	Delivery Region Index	Product Description Index	Order Quantity
0	SO - 000225	2014-01-01	126	Wholesale	USD	AXW291	364	27	
1	SO - 0003378	2014-01-01	96	Distributor	USD	AXW291	488	20	1
2	SO - 0005126	2014-01-01	8	Wholesale	USD	AXW291	155	26	
3	SO - 0005614	2014-01-01	42	Export	USD	AXW291	473	7	
4	SO - 0005781	2014-01-01	73	Wholesale	USD	AXW291	256	8	

– df_customers head –

	Customer Index	Customer Names
0	1	Geiss Company
1	2	Jaxbean Group
2	3	Ascend Ltd
3	4	Eire Corp
4	5	Blogtags Ltd

– df_products head –

	Index	Product Name
0	1	Product 1
1	2	Product 2
2	3	Product 3
3	4	Product 4
4	5	Product 5

– df_regions head –

	id	name	county	state_code	state	type	latitude	longitude	area_code	population
0	1	Auburn	Lee County	AL	Alabama	City	32.60986	-85.48078	334	620
1	2	Birmingham	Shelby County/Jefferson County	AL	Alabama	City	33.52744	-86.79905	205	212
2	3	Decatur	Limestone County/Morgan County	AL	Alabama	City	34.57332	-86.99214	256	55
3	4	Dothan	Dale County/Houston County/Henry County	AL	Alabama	City	31.23370	-85.40682	334	68

	id	name	county	state_code	state	type	latitude	longitude	area_code	populat
4	5	Hoover	Shelby County/Jefferson County	AL	Alabama	City	33.37695	-86.80558	205	841

– df_state_reg head –

	Column1	Column2	Column3
0	State Code	State	Region
1	AL	Alabama	South
2	AR	Arkansas	South
3	AZ	Arizona	West
4	CA	California	West

– df_budgets head –

	Product Name	2017 Budgets
0	Product 1	3016489.209
1	Product 2	3050087.565
2	Product 3	2642352.432
3	Product 4	2885560.824
4	Product 5	3925424.542

In [7]:

```
new_header = df_state_reg.iloc[0] # Prend la première ligne comme header
df_state_reg.columns = new_header # Remplace les noms des colonnes
df_state_reg = df_state_reg[1:].reset_index(drop=True) # Supprime la première ligne
df_state_reg.head()
```

Out[7]:

	State Code	State	Region
0	AL	Alabama	South
1	AR	Arkansas	South
2	AZ	Arizona	West
3	CA	California	West
4	CO	Colorado	West

**** Data Cleaning ****

In [8]:

```
# 🛠️ 4. DATA WRANGLING & MERGING

# 4.1 Merge with Customers
df = df_sales.merge(
    df_customers,
    how='left',
```

```

    left_on='Customer Name Index',
    right_on='Customer Index'
)

# 4.2 Merge with Products
df = df.merge(
    df_products,
    how='left',
    left_on='Product Description Index',
    right_on='Index'
)

# 4.3 Merge with Regions
df = df.merge(
    df_regions,
    how='left',
    left_on='Delivery Region Index',
    right_on='id'
)

# 4.4 Merge with State Regions
df = df.merge(
    df_state_reg[["State Code", "Region"]],
    how='left',
    left_on='state_code',
    right_on='State Code'
)

# 4.5 Merge with Budgets
df = df.merge(
    df_budgets,
    how='left',
    on='Product Name'
)

# 4.6 Clean up redundant columns
cols_to_drop = [
    'Customer Index', 'Index', 'id', 'State Code',
    # add any auto-generated cols you don't need
]
df = df.drop(columns=cols_to_drop, errors='ignore')

df.head()

```

Out[8]:

	OrderNumber	OrderDate	Customer Name Index	Channel	Currency Code	Warehouse Code	Delivery Region Index	Product Description Index	Order Quantit
0	SO - 000225	2014-01-01	126	Wholesale	USD	AXW291	364	27	
1	SO - 0003378	2014-01-01	96	Distributor	USD	AXW291	488	20	1
2	SO - 0005126	2014-01-01	8	Wholesale	USD	AXW291	155	26	
3	SO - 0005614	2014-01-01	42	Export	USD	AXW291	473	7	

	OrderNumber	OrderDate	Customer Name Index	Channel	Currency Code	Warehouse Code	Delivery Region Index	Product Description Index	Order Quantity
4	SO - 0005781	2014-01-01	73	Wholesale	USD	AXW291	256	8	

5 rows × 30 columns

In [9]:

```
df.isnull().sum()
```

Out[9]:

```
OrderNumber          0
OrderDate            0
Customer Name Index  0
Channel              0
Currency Code        0
Warehouse Code       0
Delivery Region Index 0
Product Description Index 0
Order Quantity       0
Unit Price           0
Line Total           0
Total Unit Cost      0
Customer Names       0
Product Name         0
name                 0
county               0
state_code           0
state                0
type                 0
latitude             0
longitude            0
area_code            0
population           0
households           0
median_income        0
land_area            0
water_area           0
time_zone            0
Region               0
2017 Budgets        0
dtype: int64
```

In [10]:

```
df.duplicated().sum()
```

Out[10]:

0

In [11]:

```
df.to_csv('file.csv', sep=';', index=False)
```

In [12]:

```
cols_to_drop = ['Customer Index', 'Index', 'id', 'State Code']
df = df.drop(columns = cols_to_drop , errors = 'ignore')
df.head()
```

Out[12]:

	OrderNumber	OrderDate	Customer Name Index	Channel	Currency Code	Warehouse Code	Delivery Region Index	Product Description Index	Order Quantity
0	SO - 000225	2014-01-01	126	Wholesale	USD	AXW291	364	27	
1	SO - 0003378	2014-01-01	96	Distributor	USD	AXW291	488	20	1
2	SO - 0005126	2014-01-01	8	Wholesale	USD	AXW291	155	26	
3	SO - 0005614	2014-01-01	42	Export	USD	AXW291	473	7	
4	SO - 0005781	2014-01-01	73	Wholesale	USD	AXW291	256	8	

5 rows × 30 columns

In [13]:

```
df.columns = df.columns.str.lower()
```

In [14]:

```
df.columns.values
```

Out[14]:

```
array(['ordernumber', 'orderdate', 'customer name index', 'channel',
      'currency code', 'warehouse code', 'delivery region index',
      'product description index', 'order quantity', 'unit price',
      'line total', 'total unit cost', 'customer names', 'product name',
      'name', 'county', 'state_code', 'state', 'type', 'latitude',
      'longitude', 'area_code', 'population', 'households',
      'median_income', 'land_area', 'water_area', 'time_zone', 'region',
      '2017 budgets'], dtype=object)
```

In [15]:

```
df = df.rename(columns={"county": "country"})
```

In [16]:

```
#keep the importants cols (just for now we can use another column later who knows)
cols_to_keep = [
    'ordernumber',
    'orderdate',
    'channel',
    'customer names',
    'product name',
    'order quantity',
    'unit price',
    'line total',
    'total unit cost',
    'state_code',
    'country',
    'state',
    'region',
    'latitude',
    'longitude',
    '2017 budgets'
```

```
]
```

In [17]:

```
df = df[cols_to_keep]
df.head()
```

Out[17]:

	ordernumber	orderdate	channel	customer names	product name	order quantity	unit price	line total	total unit cost	state_
0	SO - 000225	2014-01-01	Wholesale	Rhynoodle Ltd	Product 27	6	2499.1	14994.6	1824.343	
1	SO - 0003378	2014-01-01	Distributor	Thoughtmix Ltd	Product 20	11	2351.7	25868.7	1269.918	
2	SO - 0005126	2014-01-01	Wholesale	Amerisourc Corp	Product 26	6	978.2	5869.2	684.740	
3	SO - 0005614	2014-01-01	Export	Colgate-Pa Group	Product 7	7	2338.3	16368.1	1028.852	
4	SO - 0005781	2014-01-01	Wholesale	Deseret Group	Product 8	8	2291.4	18331.2	1260.270	

In [18]:

```
# Rename to more Pythonic / self-explanatory names, including region
df = df.rename(columns={
    'ordernumber'      : 'order_number',
    'orderdate'        : 'order_date',
    'customer names'   : 'customer_name',
    'product name'     : 'product_name',
    'order quantity'   : 'quantity',
    'unit price'       : 'unit_price',
    'line total'       : 'revenue',
    'total unit cost'  : 'cost',
    'state_code'       : 'state',
    'state'            : 'state_name',
    'region'           : 'us_region',
    'latitude'         : 'lat',
    'longitude'        : 'lon',
    '2017 budgets'     : 'budget'
})
df.head()
```

Out[18]:

	order_number	order_date	channel	customer_name	product_name	quantity	unit_price	revenue
0	SO - 000225	2014-01-01	Wholesale	Rhynoodle Ltd	Product 27	6	2499.1	14994.6
1	SO - 0003378	2014-01-01	Distributor	Thoughtmix Ltd	Product 20	11	2351.7	25868.7
2	SO - 0005126	2014-01-01	Wholesale	Amerisourc Corp	Product 26	6	978.2	5869.2
3	SO - 0005614	2014-01-01	Export	Colgate-Pa Group	Product 7	7	2338.3	16368.1
4	SO - 0005781	2014-01-01	Wholesale	Deseret Group	Product 8	8	2291.4	18331.2

order_number	order_date	channel	customer_name	product_name	quantity	unit_price	revenue
--------------	------------	---------	---------------	--------------	----------	------------	---------

In [19]:

```
# Blank out budgets for non-2017 orders
df.loc[df['order_date'].dt.year != 2017, 'budget'] = pd.NA

# Inspect
df[['order_date', 'product_name', 'revenue', 'budget']].head(10)
```

Out[19]:

	order_date	product_name	revenue	budget
0	2014-01-01	Product 27	14994.6	NaN
1	2014-01-01	Product 20	25868.7	NaN
2	2014-01-01	Product 26	5869.2	NaN
3	2014-01-01	Product 7	16368.1	NaN
4	2014-01-01	Product 8	18331.2	NaN
5	2014-01-01	Product 23	57412.3	NaN
6	2014-01-01	Product 1	20120.1	NaN
7	2014-01-01	Product 25	10452.0	NaN
8	2014-01-01	Product 13	19262.5	NaN
9	2014-01-01	Product 14	34974.0	NaN

In [20]:

```
df.isnull().sum()
```

Out[20]:

order_number	0
order_date	0
channel	0
customer_name	0
product_name	0
quantity	0
unit_price	0
revenue	0
cost	0
state	0
country	0
state_name	0
us_region	0
lat	0
lon	0
budget	48841

dtype: int64

In [21]:

```
df.duplicated().sum()
```

Out[21]:

0

🔧 Feature Engineering

In [22]:

```
#1 calculate total cost for each line item
df['total_cost'] = df['quantity']* df['cost']
df['total_cost'].head()
```

Out[22]:

```
0    10946.058
1    13969.098
2     4108.440
3     7201.964
4    10082.160
Name: total_cost, dtype: float64
```

In [23]:

```
df['profit'] = df['revenue'] - df['total_cost']
df['profit'].head()
```

Out[23]:

```
0     4048.542
1    11899.602
2     1760.760
3     9166.136
4     8249.040
Name: profit, dtype: float64
```

In [24]:

```
df['profit_marge_pct'] = (df['profit']/df['revenue'])*100
df['profit_marge_pct'].head()
```

Out[24]:

```
0     27.0
1     46.0
2     30.0
3     56.0
4     45.0
Name: profit_marge_pct, dtype: float64
```

In [25]:

```
#extract full month name from order_date
df['order_month_name'] = df['order_date'].dt.month_name()
df['order_month_name'].sort_values().unique()
```

Out[25]:

```
array(['April', 'August', 'December', 'February', 'January', 'July',
      'June', 'March', 'May', 'November', 'October', 'September'],
      dtype=object)
```

In [26]:

```
# 5. Extract month number from order_date for correct sorting (1-12)
df['order_month_num'] = df['order_date'].dt.month
```

In [27]:

```
df.head()
```

Out[27]:

	order_number	order_date	channel	customer_name	product_name	quantity	unit_price	revenue
0	SO - 000225	2014-01-01	Wholesale	Rhynoodle Ltd	Product 27	6	2499.1	14994.6
1	SO - 0003378	2014-01-01	Distributor	Thoughtmix Ltd	Product 20	11	2351.7	25868.7
2	SO - 0005126	2014-01-01	Wholesale	Amerisourc Corp	Product 26	6	978.2	5869.2
3	SO - 0005614	2014-01-01	Export	Colgate-Pa Group	Product 7	7	2338.3	16368.1
4	SO - 0005781	2014-01-01	Wholesale	Deseret Group	Product 8	8	2291.4	18331.2

5 rows × 21 columns

In [28]:

```
df.columns
```

Out[28]:

```
Index(['order_number', 'order_date', 'channel', 'customer_name',
      'product_name', 'quantity', 'unit_price', 'revenue', 'cost', 'state',
      'country', 'state_name', 'us_region', 'lat', 'lon', 'budget',
      'total_cost', 'profit', 'profit_marge_pct', 'order_month_name',
      'order_month_num'],
      dtype='object')
```



Exploratory Analysis

◆ 1. Monthly Sales Trend Over Time

Objectif : Suivre l'évolution du chiffre d'affaires dans le temps pour détecter la saisonnalité ou les pics de ventes

Graphique : line chart

Type d'EDA : Temporel

In [29]:

```
# Convert order_date to monthly period (e.g., 2022-01, 2022-02)
df['order_month'] = df['order_date'].dt.to_period('M')

# Calculate total revenue for each month
monthly_sales = df.groupby('order_month')['revenue'].sum()

# Set figure size for clarity
plt.figure(figsize=(15,4))

# Plot the monthly sales trend with circle markers and navy line
monthly_sales.plot(marker='o', color='navy')

# Scale y-axis values to millions for readability
from matplotlib.ticker import FuncFormatter
formatter = FuncFormatter(lambda x, pos: f'{x/1e6:.1f}M')
plt.gca().yaxis.set_major_formatter(formatter)

# Add title and axis labels
plt.title('Monthly Sales Trend')
plt.xlabel('Month')
```

```
plt.ylabel('Total Revenue (Millions)')

# Rotate x-axis labels for better readability
plt.xticks(rotation=45)

# Adjust layout to prevent clipping
plt.tight_layout()
plt.show()
```



INSIGHTS

Les ventes varient régulièrement entre 24 M et 26 M, avec des pics au printemps/été (mai-juin) et des creux en janvier.

La tendance générale reste stable chaque année, montrant un schéma de demande saisonnière fiable.

Cependant, la forte baisse des ventes début 2017 est un cas particulier qui mérite d'être analysé, par exemple à cause de perturbations du marché ou de promotions mal programmées.

♦ 2. Tendance des ventes mensuelles (tous les ans combinés)

Objectif : Mettre en évidence les motifs saisonniers en agrégeant les ventes de toutes les années pour chaque mois du calendrier

Graphique : line chart

Type d'EDA : Temporel (séries temporelles)

In [30]:

```
# 1. Filter out any 2018 orders
df_ = df[df['order_date'].dt.year != 2018] # exclude any partial-year data (i.e., Janu
#I create df_ so I don't alter the original

# 2. Group by month number and month name, sum revenue, then sort by month number
monthly_sales = (
    df_
    .groupby(['order_month_num', 'order_month_name'])['revenue']
    .sum()
    .sort_index()
)

# 3. Plot setup
from matplotlib.ticker import FuncFormatter
```

```
plt.figure(figsize=(13, 4))
plt.plot(
    monthly_sales.index.get_level_values(1), # X-axis: month names
    monthly_sales.values,                   # Y-axis: total revenue
    marker='o',                             # circle markers
    color='navy'                            # line color
)

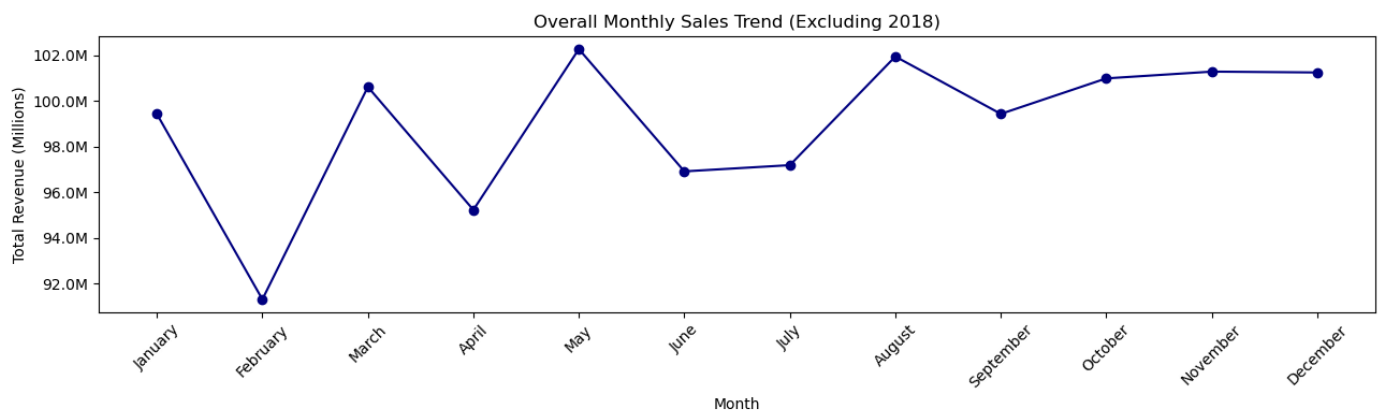
# 4. Scale y-axis values to millions for readability
formatter = FuncFormatter(lambda x, pos: f'{x/1e6:.1f}M')
plt.gca().yaxis.set_major_formatter(formatter)

# 5. Add title and axis labels
plt.title('Overall Monthly Sales Trend (Excluding 2018)')
plt.xlabel('Month')
plt.ylabel('Total Revenue (Millions)')

# 6. Rotate x-axis labels for readability
plt.xticks(rotation=45)

# 7. Adjust layout to prevent clipping
plt.tight_layout()

# 8. Display the plot
plt.show()
```



INSIGHTS --

-Février : le mois le plus faible (92 M) Avril–Mai : les meilleurs mois (pic à ~102 M) Juin–Juillet : forte baisse
Août à Décembre : très stable et assez haut (autour de 99–100 M)

En résumé : L'année commence bas, explose au printemps, redescend en été, puis reste élevée et stable de août jusqu'à Noël.

♦ 3. Top 10 Products by Revenue (in Millions)

Objectif : Identifier les produits les plus rentables pour concentrer les efforts de marketing et d'inventaire.

Chart: Horizontal bar chart

In [53]:

```
# Calculate total revenue for each product and convert values to millions
top_prod = df.groupby('product_name')['revenue'].sum() / 1_000_000
```

```
# Select the top 10 products by revenue
top_prod = top_prod.nlargest(10)

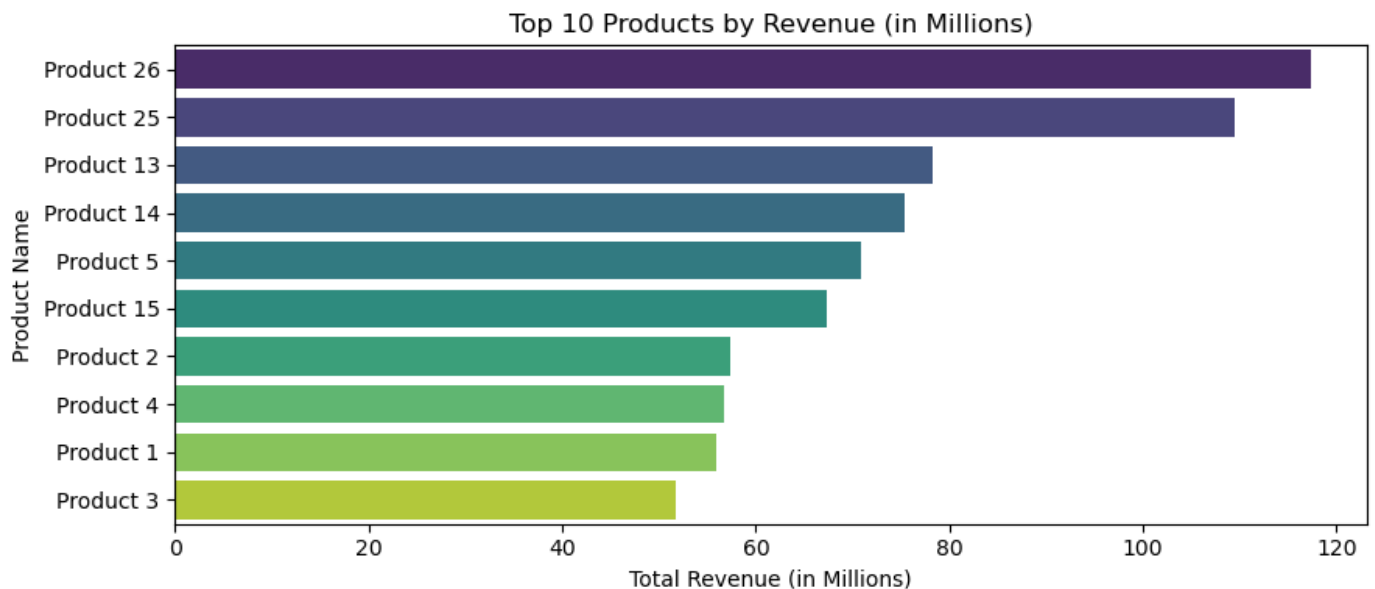
# Set the figure size for clarity
plt.figure(figsize=(9, 4))

# Plot a horizontal bar chart: x-axis as revenue in millions, y-axis as product names
sns.barplot(
    x=top_prod.values,      # X-axis: revenue values in millions
    y=top_prod.index,      # Y-axis: product names
    palette='viridis'      # Color palette for bars
)

# Add title and axis labels
plt.title('Top 10 Products by Revenue (in Millions)') # Main title
plt.xlabel('Total Revenue (in Millions)')           # X-axis label
plt.ylabel('Product Name')                          # Y-axis label

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Display the plot
plt.show()
```



INSIGHTS

Product 26 → le champion absolu (~115 M)

Product 25 → 2^e place, très proche (~105 M) Les places 3 à 9 sont moyennes (entre 40 et 65 M). Le 10^e (Product 3) est vraiment faible.

♦ 4. Top 10 Products by Avg Profit Margin

Objectif : Comparer la rentabilité moyenne des produits pour identifier ceux avec les marges les plus élevées

Graphique : Diagramme à barres horizontales

In [37]:

```

# Calculate total revenue for each product and convert values to millions
top_prod = df.groupby('product_name')['revenue'].sum() / 1_000_000

# Select the top 10 products by revenue
top_prod = top_prod.nlargest(10)

# Set the figure size for clarity
plt.figure(figsize=(9, 4))

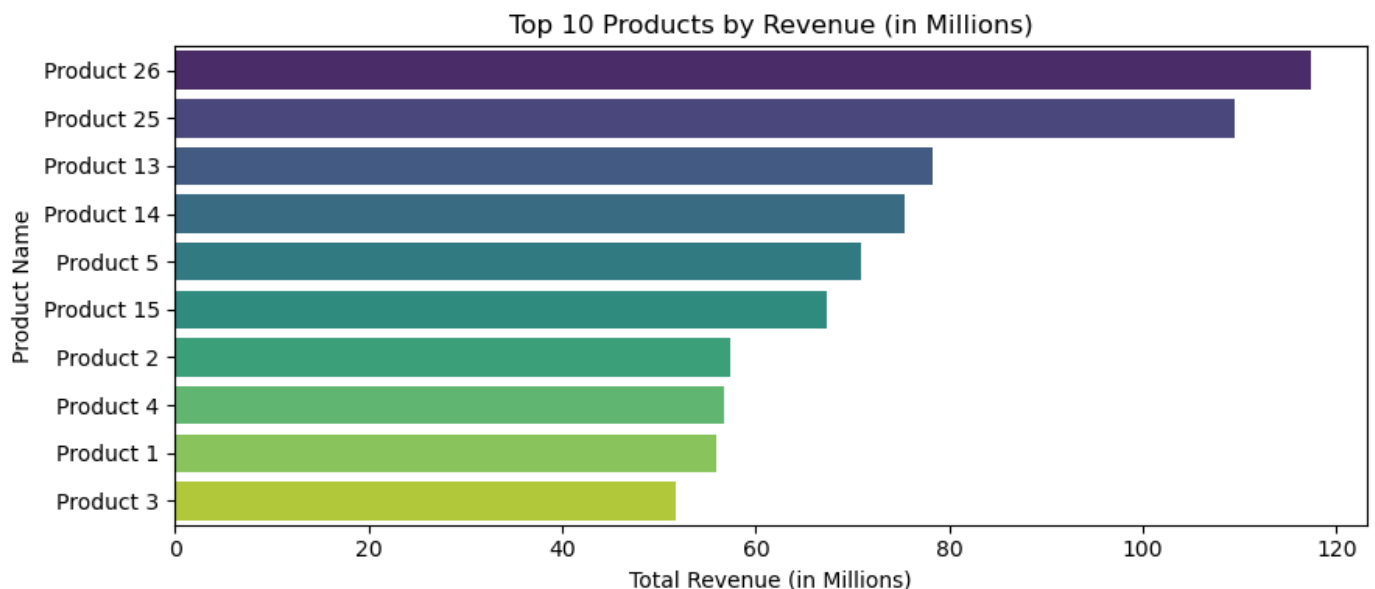
# Plot a horizontal bar chart: x-axis as revenue in millions, y-axis as product names
sns.barplot(
    x=top_prod.values,      # X-axis: revenue values in millions
    y=top_prod.index,       # Y-axis: product names
    palette='viridis'       # Color palette for bars
)

# Add title and axis labels
plt.title('Top 10 Products by Revenue (in Millions)') # Main title
plt.xlabel('Total Revenue (in Millions)')           # X-axis label
plt.ylabel('Product Name')                          # Y-axis label

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Display the plot
plt.show()

```



INSIGHTS

Les produits 18 et 28 dominent avec des marges bénéficiaires moyennes d'environ 8 000 à 8 300.

Viennent ensuite les produits 5 et 11, avec des marges proches de 7 900 à 8 000.

Des produits de performance moyenne comme les produits 12, 26 et 21 ont des marges autour de 7 700 à 7 800.

Enfin, les produits 4, 16 et 1 se situent entre 7 400 et 7 600.

◆ 5. Sales by Channel (Pie Chart)

Objectif : Montrer la répartition des ventes totales par canal pour identifier les canaux de vente dominants

Graphique : pie chart

In [38]:

```
# Group revenue by sales channel and sort descending
chan_sales = df.groupby('channel')['revenue'].sum().sort_values(ascending=False)

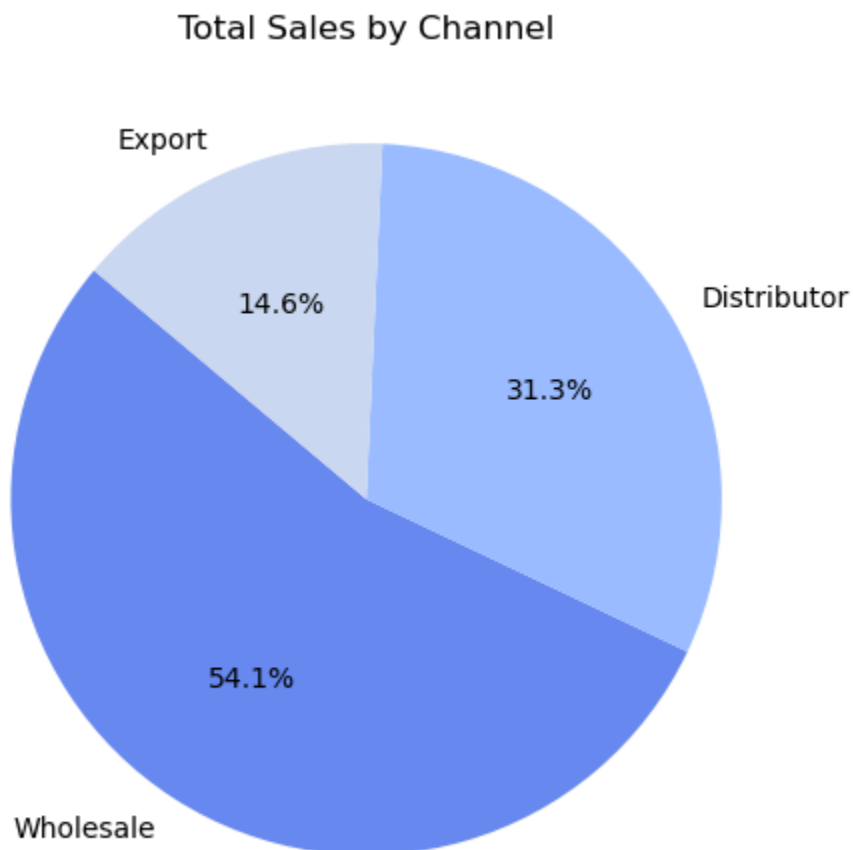
# Set figure size for the pie chart
plt.figure(figsize=(5, 5))

# Plot pie chart with percentage labels and a defined start angle
plt.pie(
    chan_sales.values,          # Data: revenue values per channel
    labels=chan_sales.index,    # Labels: channel names
    autopct='%1.1f%%',         # Display percentages with one decimal
    startangle=140,             # Rotate chart so first slice starts at 140 deg
    colors=sns.color_palette('coolwarm') # Color palette for slices
)

# Add title for context
plt.title('Total Sales by Channel')

# Adjust layout to ensure everything fits well
plt.tight_layout()

# Display the chart
plt.show()
```



INSIGHTS

Le canal de vente en gros représente 54 % des ventes, suivi par les distributeurs à ~31 % et les exportations à ~15 %, ce qui montre une dépendance importante aux canaux de vente domestiques.

Pour diversifier les revenus et réduire le risque de concentration, il est conseillé de prioriser l'expansion des initiatives d'exportation, en mettant l'accent sur le marketing ciblé à l'international et les relations stratégiques avec des partenaires.

◆ 6. Average Order Value (AOV) Distribution

Objectif : Comprendre la distribution des valeurs des commandes pour identifier les niveaux typiques de dépenses et les valeurs aberrantes

Graphique : Histogramme

In [40]:

```
# Calculate the total revenue for each order to get the order value
aov = df.groupby('order_number')['revenue'].sum()

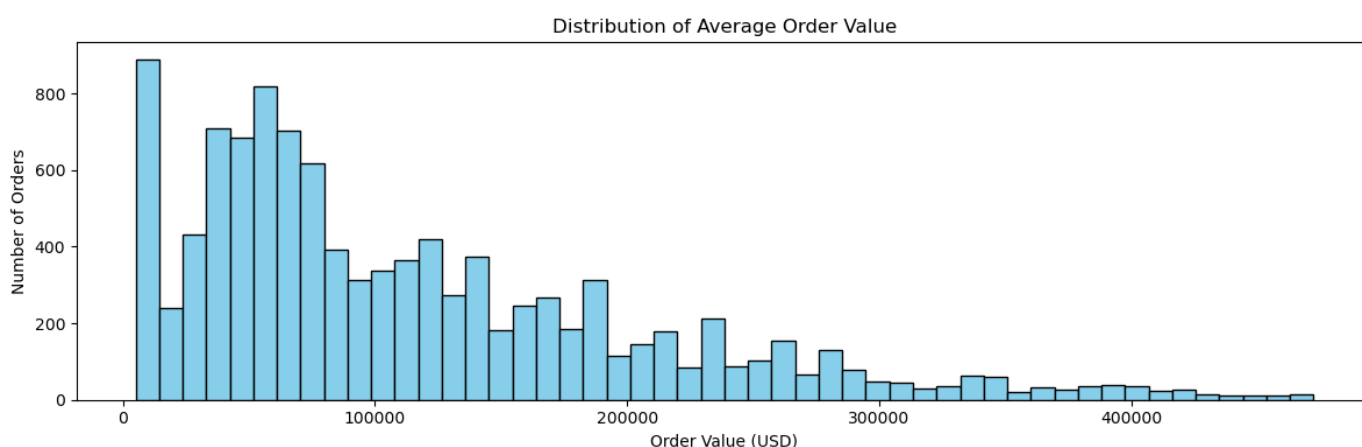
# Set the figure size for better visibility
plt.figure(figsize=(12, 4))

# Plot a histogram of order values
plt.hist(
    aov,                # Data: list of order values
    bins=50,           # Number of bins to group order values
    color='skyblue',    # Fill color of the bars
    edgecolor='black'   # Outline color of the bars
)

# Add title and axis labels for context
plt.title('Distribution of Average Order Value')
plt.xlabel('Order Value (USD)')
plt.ylabel('Number of Orders')

# Adjust layout to prevent clipping
plt.tight_layout()

# Show the plot
plt.show()
```



INSIGHTS

La distribution de la valeur des commandes est fortement asymétrique à droite, avec la majorité des commandes regroupées entre 120 K et un mode prononcé autour de 60 K.

Une longue queue de transactions à haute valeur s'étend jusqu'à 500 K, mais ces grandes commandes représentent seulement une petite part du volume total.

♦ 7. Profit Margin % vs. Unit Price

Objectif : Examiner la relation entre le prix unitaire et le pourcentage de la marge bénéficiaire à travers les commandes

Graphique : scatter plot

In [41]:

```
# Calculate profit margin percentage for each record
df['profit_margin_pct'] = (df['profit'] / df['revenue']) * 100

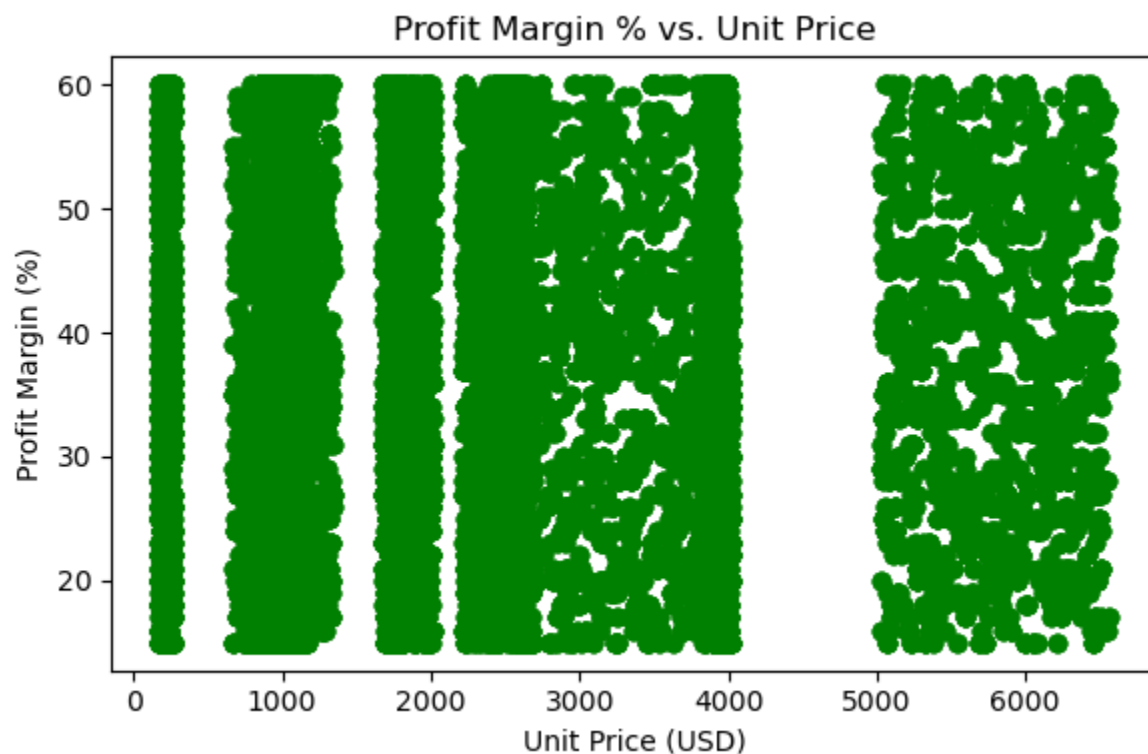
# Set the figure size for clarity
plt.figure(figsize=(6,4))

# Plot unit price vs. profit margin percentage
plt.scatter(
    df['unit_price'],          # X-axis: unit price in USD
    df['profit_margin_pct'],   # Y-axis: profit margin percentage
    alpha=0.6,                # Transparency level for overlapping points
    color='green'              # Point color
)

# Add title and axis labels
plt.title('Profit Margin % vs. Unit Price') # Chart title
plt.xlabel('Unit Price (USD)')              # X-axis label
plt.ylabel('Profit Margin (%)')             # Y-axis label

# Adjust layout to prevent clipping
plt.tight_layout()

# Display the plot
plt.show()
```



INSIGHTS

Les marges bénéficiaires sont concentrées entre ~18 % et 60 %, sans corrélation claire avec le prix unitaire, qui varie à partir de près de 6 500 USD.

Des bandes horizontales denses indiquent des niveaux de marge constants à travers un large éventail de prix, reflétant des stratégies de tarification uniformes.

Les valeurs aberrantes sous 18 % aux deux points de prix faible et élevé peuvent signaler des inefficacités de coûts ou des problèmes de tarification, méritant une analyse plus approfondie.

♦ 8. Distribution des Prix Unitaires par Produit

Objectif : Comparer la variabilité des prix entre les produits pour identifier la cohérence des prix et les valeurs aberrantes

Graphique : Boxplot

In [42]:

```
# Set figure size for clarity
plt.figure(figsize=(12,4))

# Create a boxplot of unit_price by product_name
sns.boxplot(
    data=df,
    x='product_name',    # X-axis: product categories
    y='unit_price',      # Y-axis: unit price values
    color='g'           # Box color
)

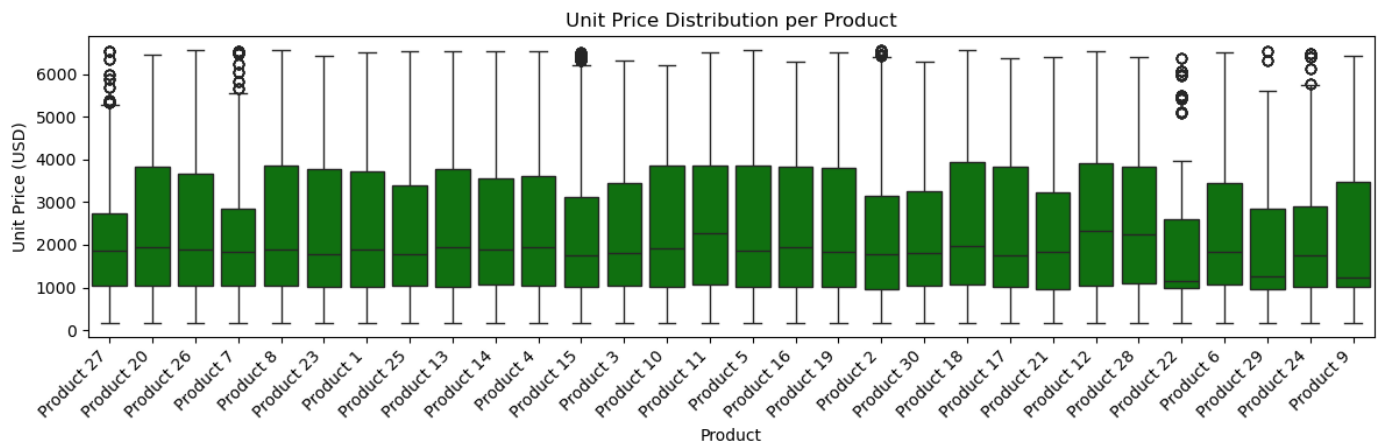
# Add title and axis labels
plt.title('Unit Price Distribution per Product') # Chart title
```

```
plt.xlabel('Product') # X-axis label
plt.ylabel('Unit Price (USD)') # Y-axis label

# Rotate x-axis labels for better readability
plt.xticks(rotation=45, ha='right')

# Adjust layout to prevent clipping of labels
plt.tight_layout()

# Display the plot
plt.show()
```



INSIGHTS

Les produits 8, 17, 27, 20 et 28 montrent des pics de revenus élevés, bien au-dessus de leurs whiskers supérieurs, probablement en raison de commandes en gros, de sorties en édition spéciale, ou de bundles premium qui gonflent temporairement les revenus.

À l'inverse, les produits 20 et 27 montrent de profonds bas de gamme, suggérant des cadeaux promotionnels ou des SKU test qui tirent les prix moyens vers le bas.

♦ 9. Ventes Totales par Région des USA

Objectif : Comparer les ventes totales selon les régions des USA pour identifier les marchés les plus performants et les zones à développer

Graphique : Diagramme à barres horizontales

In [43]:

```
# 1. Aggregate total sales by region (in millions)
region_sales = (
    df
    .groupby('us_region')['revenue']
    .sum()
    .sort_values(ascending=False) # so bars go top-bottom
    / 1e6 # convert to millions
)

# 2. Plot
plt.figure(figsize=(10, 4))
sns.barplot(
    x=region_sales.values,
```

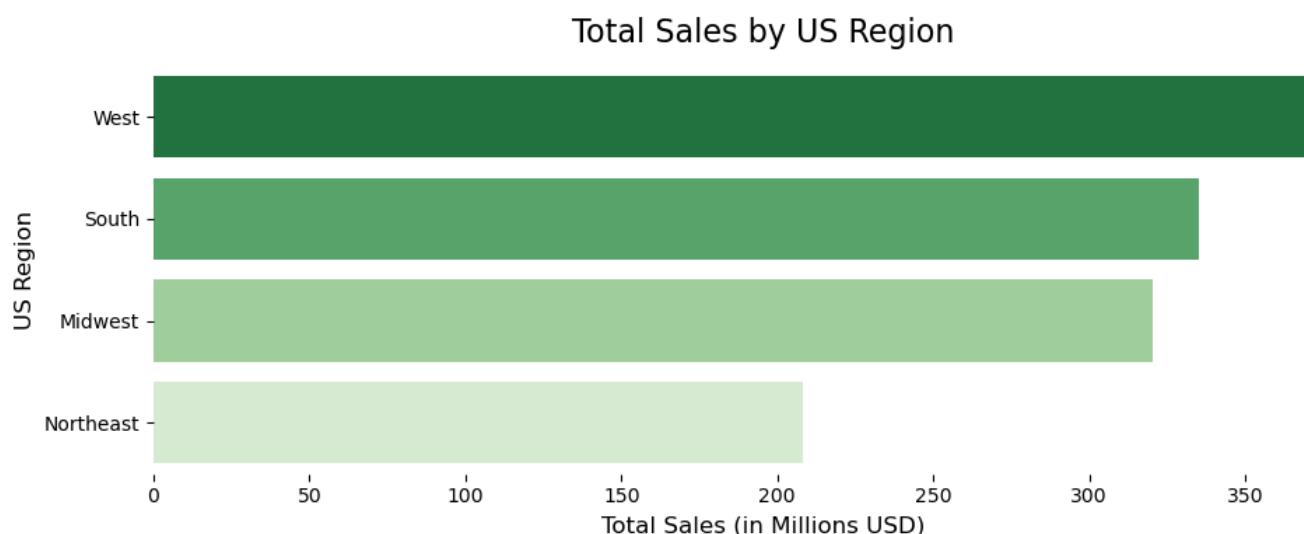
```

y=region_sales.index,
palette='Greens_r'          # dark→light green
)

# 3. Formatting
plt.title('Total Sales by US Region', fontsize=16, pad=12)
plt.xlabel('Total Sales (in Millions USD)', fontsize=12)
plt.ylabel('US Region', fontsize=12)
plt.xticks(rotation=0)
sns.despine(left=True, bottom=True)

plt.tight_layout()
plt.show()

```



INSIGHTS

L'Ouest domine avec environ 360 M USD de ventes (~35 % du total), ce qui souligne sa position de leader sur le marché.

Le Sud et le Midwest génèrent chacun plus de 320 M USD (~32 %), indiquant une demande forte et stable dans les régions centrales.

Le Northeast suit avec environ 210 M USD (~20 %), montrant un potentiel de croissance et un besoin d'investissement ciblé.

Action : Se concentrer sur la réduction de l'écart avec le Northeast grâce à des promotions locales et des partenariats stratégiques, tout en maintenant le succès national.

♦ 10. Ventes Totales par État

Objectif : Visualiser la répartition géographique des ventes pour identifier les états à forte et faible performance et découvrir les écarts régionaux

Graphique : Carte choroplèthe des USA

In [44]:

```
import plotly.express as px

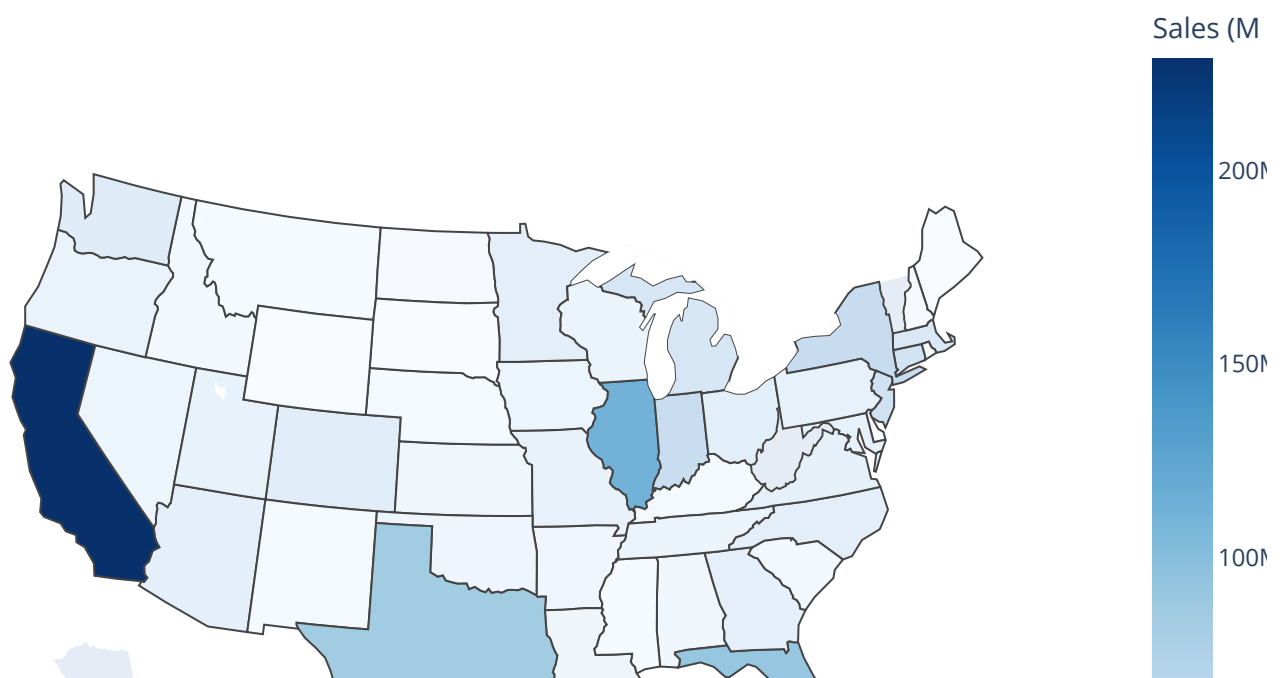
# 1. Aggregate revenue by state (in millions)
state_sales = (
    df
    .groupby('state')['revenue']
    .sum()
    .reset_index()
)
state_sales['revenue_m'] = state_sales['revenue'] / 1e6 # convert to millions

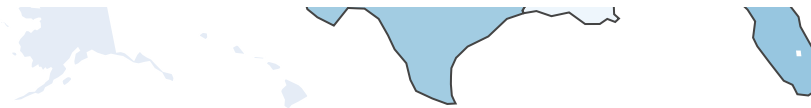
# 2. Plotly choropleth
fig = px.choropleth(
    state_sales,
    locations='state',          # column with state codes
    locationmode='USA-states',  # tells Plotly these are US states
    color='revenue_m',
    scope='usa',
    labels={'revenue_m': 'Total Sales (M USD)'},
    color_continuous_scale='Blues',
    hover_data={'revenue_m': ':.2f'} # show 2 decimals
)

# 3. Layout tuning
fig.update_layout(
    title_text='Total Sales by State',
    margin=dict(l=0, r=0, t=40, b=0),
    coloraxis_colorbar=dict(
        title='Sales (M USD)',
        ticksuffix='M'
    )
)

fig.show()
```

Total Sales by State





50M

INSIGHTS

Californie mène largement avec 230 M USD de revenus et 7 500 commandes, plus du double des états suivants.

Illinois, Floride et Texas forment une solide deuxième couche, avec des revenus entre 85–110 M USD et 3 600–4 600 commandes.

New York et Indiana sont au milieu avec des revenus de 35–44 M USD et des commandes de 1 600–2 100.

♦ 11. Top 10 des Clients par Revenu et Nombre de Commandes

Objectif : Identifier les clients générant le plus de revenus et comparer leurs volumes de commandes

Graphique : Deux graphiques à barres

In [45]:

```
#  Calculate total revenue per customer and select top 10
top_rev = (
    df.groupby('customer_name')['revenue']
      .sum()                      # Sum revenue for each customer
      .sort_values(ascending=False) # Sort from highest to lowest
      .head(10)                   # Keep top 10 customers
)

#  Calculate total revenue per customer and select bottom 10
bottom_rev = (
    df.groupby('customer_name')['revenue']
      .sum()                      # Sum revenue for each customer
      .sort_values(ascending=True) # Sort from lowest to highest
      .head(10)                   # Keep bottom 10 customers
)

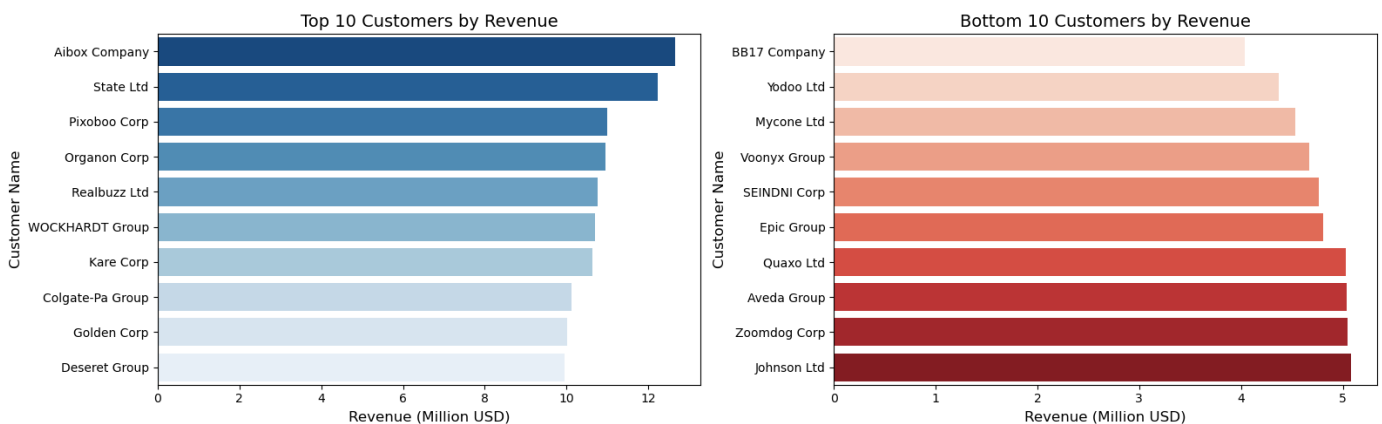
# Create a figure with two side-by-side subplots
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Plot 1: Top 10 customers by revenue (converted to millions)
sns.barplot(
    x=top_rev.values / 1e6,      # X-axis: revenue in millions
    y=top_rev.index,             # Y-axis: customer names
    palette='Blues_r',           # Color palette (reversed blues)
    ax=axes[0]                   # Draw on the left subplot
)

axes[0].set_title('Top 10 Customers by Revenue', fontsize=14)           # Title
axes[0].set_xlabel('Revenue (Million USD)', fontsize=12)                # X-axis label
axes[0].set_ylabel('Customer Name', fontsize=12)                        # Y-axis label
```

```
# Plot 2: Bottom 10 customers by revenue (converted to millions)
sns.barplot(
    x=bottom_rev.values / 1e6, # X-axis: revenue in millions
    y=bottom_rev.index,        # Y-axis: customer names
    palette='Reds',            # Color palette (reds)
    ax=axes[1]                 # Draw on the right subplot
)
axes[1].set_title('Bottom 10 Customers by Revenue', fontsize=14) # Title
axes[1].set_xlabel('Revenue (Million USD)', fontsize=12)         # X-axis label
axes[1].set_ylabel('Customer Name', fontsize=12)                # Y-axis label

# Adjust layout to prevent overlap and display both charts
plt.tight_layout()
plt.show()
```



INSIGHTS

Aibox Company arrive en tête avec 12.2 M USD, tandis que Deseret Group, 10e, génère entre 10–12 M USD.

À l'autre extrémité, Johnson Ltd génère 4.1 M USD, soit environ la moitié des revenus du client numéro un.

Cette chute importante entre 4–5 M USD met en évidence une concentration élevée des revenus chez les meilleurs clients.

Action : Prioriser la rétention et les upsell pour les 10 premiers clients, et lancer des campagnes de croissance ciblées pour les clients à faible revenu

◆ 12. Average Profit Margin by Channel

Objectif : Comparer les marges bénéficiaires moyennes des différents canaux de vente pour identifier les routes les plus et les moins rentables

Graphique : Diagramme à barres verticales

In [46]:

```
# Compute average profit margin percentage for each channel
channel_margin = (
    df.groupby('channel')['profit_margin_pct'] # Group by sales channel
      .mean()                                # Calculate mean profit margin %
      .sort_values(ascending=False)          # Sort channels from highest to lowest ma
)
```

```

# 2 Set the figure size for clarity
plt.figure(figsize=(6, 4))

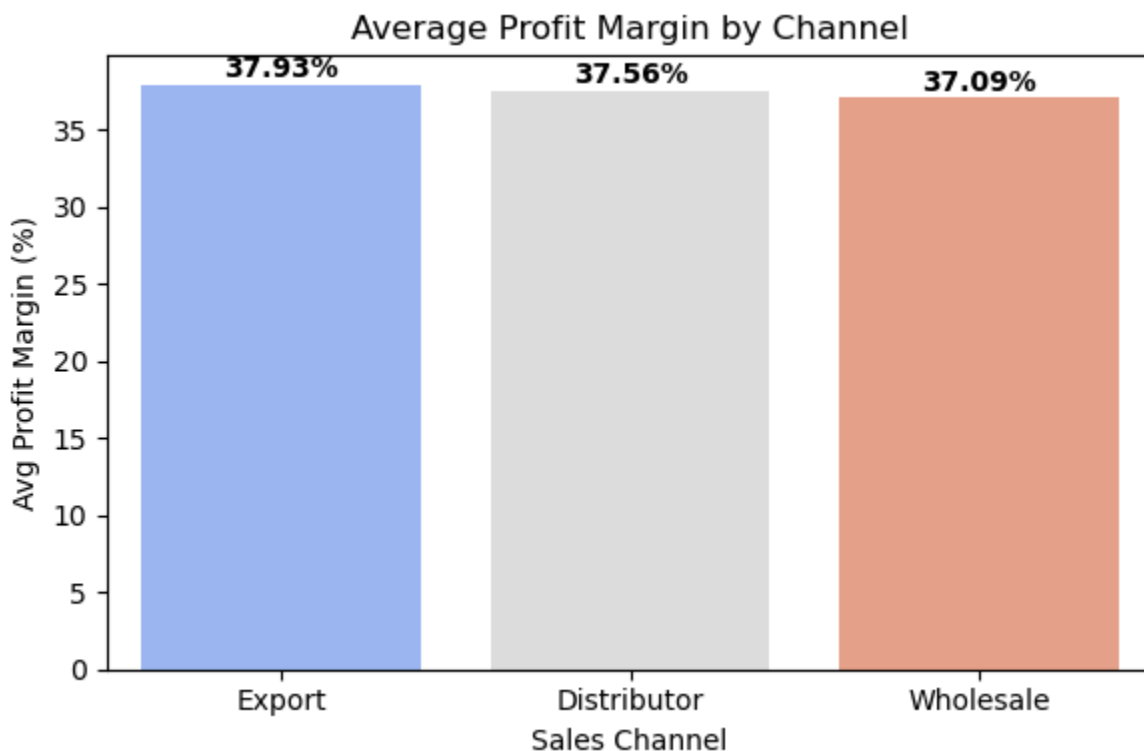
# 3 Plot a bar chart of average profit margin by channel
ax = sns.barplot(
    x=channel_margin.index,      # X-axis: channel names
    y=channel_margin.values,     # Y-axis: average profit margin values
    palette='coolwarm'          # Color palette for bars
)

# 4 Add chart title and axis labels
plt.title('Average Profit Margin by Channel') # Main title
plt.xlabel('Sales Channel')                  # X-axis label
plt.ylabel('Avg Profit Margin (%)')           # Y-axis label

# 5 Annotate each bar with its exact margin percentage
for i, v in enumerate(channel_margin.values):
    ax.text(
        i,                                # X position (bar index)
        v + 0.5,                          # Y position (bar height + small offset)
        f'{v:.2f}%',                      # Text label showing percentage with two decimals
        ha='center',                      # Center-align the text horizontally
        fontweight='bold'                 # Bold font for readability
    )

# 6 Adjust layout to prevent clipping and display the plot
plt.tight_layout()
plt.show()

```



INSIGHTS

Export mène avec une marge moyenne de 37.93 %, suivi par Distributor (37.56 %) et Wholesale (37.09 %)

L'écart minuscule (<0.2 %) montre une rentabilité uniforme à travers tous les canaux.

Cette uniformité implique des coûts bien maîtrisés et un pouvoir de tarification similaire sur tous les canaux.

Pour maximiser les retours, il faut pousser la croissance du volume dans le canal Export tout en maintenant l'efficacité dans Distributor et Wholesale.

◆ 13. Top 10 et Bottom 10 Clients par Revenu

Objectif : Identifier les clients générant le plus de revenus et ceux générant les moins, afin de personnaliser les stratégies d'engagement

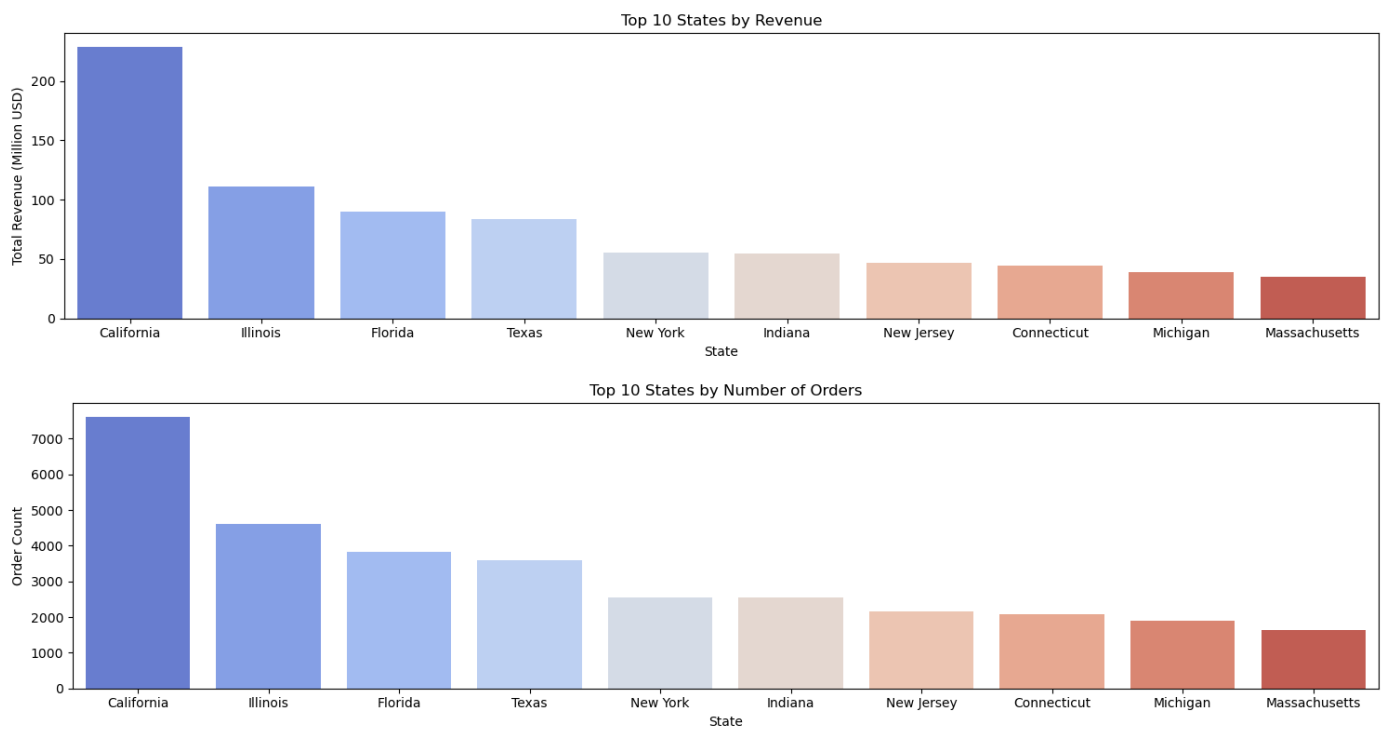
Graphique : Graphiques à barres horizontales côte à côte

In [47]:

```
# Aggregate total revenue and unique order count per state
state_rev = df.groupby('state_name').agg(
    revenue=('revenue', 'sum'),          # Sum up revenue per state
    orders=('order_number', 'nunique')   # Count unique orders per state
).sort_values('revenue', ascending=False).head(10) # Keep top 10 by revenue

# Plot 1: Top 10 states by revenue (scaled to millions)
plt.figure(figsize=(15, 4))
sns.barplot(
    x=state_rev.index,                  # X-axis: state names
    y=state_rev['revenue'] / 1e6,       # Y-axis: revenue in millions
    palette='coolwarm'                 # Color palette
)
plt.title('Top 10 States by Revenue')  # Chart title
plt.xlabel('State')                   # X-axis label
plt.ylabel('Total Revenue (Million USD)') # Y-axis label
plt.tight_layout()                    # Adjust layout
plt.show()                            # Display the plot

# Plot 2: Top 10 states by number of orders
plt.figure(figsize=(15, 4))
sns.barplot(
    x=state_rev.index,                  # X-axis: state names
    y=state_rev['orders'],               # Y-axis: order counts
    palette='coolwarm'                 # Color palette
)
plt.title('Top 10 States by Number of Orders') # Chart title
plt.xlabel('State')                     # X-axis label
plt.ylabel('Order Count')               # Y-axis label
plt.tight_layout()                     # Adjust layout
plt.show()                             # Display the plot
```



◆ 14. Customer Segmentation: Revenue vs. Profit Margin

Objectif : Segmenter les clients en fonction du revenu total et de la marge bénéficiaire moyenne, en mettant en évidence le volume des commandes

Graphique : Diagramme à bulles (graphique de dispersion avec tailles variables des points)

In [48]:

```
# Aggregate metrics per customer
cust_summary = df.groupby('customer_name').agg(
    total_revenue=('revenue', 'sum'),
    total_profit=('profit', 'sum'),
    avg_margin=('profit_margin_pct', 'mean'),
    orders=('order_number', 'nunique')
)

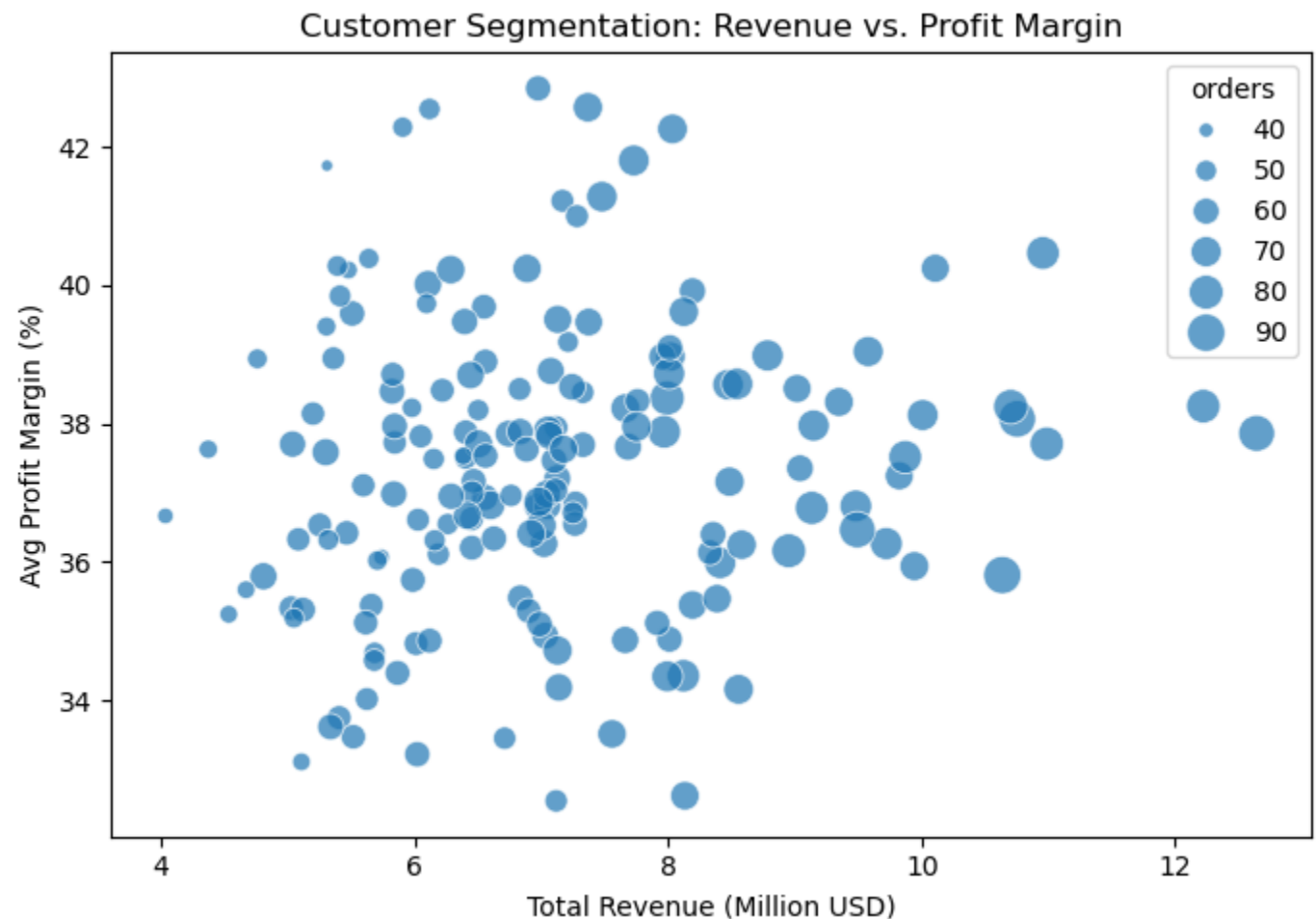
# Convert revenue to millions
cust_summary['total_revenue_m'] = cust_summary['total_revenue'] / 1e6

plt.figure(figsize=(7, 5))

# Bubble chart with revenue in millions
sns.scatterplot(
    data=cust_summary,
    x='total_revenue_m',          # <-- use revenue in millions
    y='avg_margin',
    size='orders',
    sizes=(20, 200),
    alpha=0.7
)

plt.title('Customer Segmentation: Revenue vs. Profit Margin')
plt.xlabel('Total Revenue (Million USD)') # <-- updated label
plt.ylabel('Avg Profit Margin (%)')
```

```
plt.tight_layout()
plt.show()
```



INSIGHTS

Les clients avec des revenus supérieurs à 10 M USD ont des marges stables entre 36–40 %, indiquant une rentabilité cohérente avec l'échelle.

La majorité des clients se trouvent dans la plage de 6–10 M USD, avec des marges également stables.

Les clients sous 6 M USD montrent plus de variabilité dans leurs marges (~33–43 %), ce qui peut suggérer une volatilité des coûts ou des remises.

Le volume des commandes augmente avec le revenu, mais la marge reste relativement constante, ce qui met en évidence le revenu comme facteur dominant de performance.

♦ 15. Correlation Heatmap of Numeric Features

Objectif : Identifier les relations entre les variables numériques clés pour découvrir une éventuelle multicollinéarité

Graphique : heatmap

In [49]:

```
# List numeric columns to include in the correlation calculation
num_cols = ['quantity', 'unit_price', 'revenue', 'cost', 'profit']
```

```
# Calculate the correlation matrix for these numeric features
corr = df[num_cols].corr()

# Set the figure size for clarity
plt.figure(figsize=(6,4))

# Plot the heatmap with annotations and a viridis colormap
sns.heatmap(
    corr,          # Data: correlation matrix
    annot=True,    # Display the correlation coefficients on the heatmap
    fmt=".2f",     # Format numbers to two decimal places
    cmap='viridis' # Color palette for the heatmap
)

# Add title for context
plt.title('Correlation Matrix')

# Adjust layout to prevent clipping
plt.tight_layout()

# Display the heatmap
plt.show()
```



INSIGHTS

Profit et revenu sont fortement corrélés (0.87), ce qui indique qu'à mesure que la valeur des ventes augmente, le profit tend également à augmenter.

Le prix unitaire est un facteur clé : il est corrélé à 0.91 avec le revenu, 0.79 avec le profit et 0.94 avec le coût, mettant en évidence l'impact des décisions de tarification sur les revenus et les coûts.

Le coût montre une forte corrélation avec le revenu (0.85) mais une corrélation plus modérée avec le profit (0.58), ce qui montre que bien que des ventes plus élevées entraînent souvent des dépenses plus élevées,

les marges peuvent encore varier.

La quantité n'a pratiquement aucune corrélation avec le prix unitaire ou le coût (≈ 0.00) et n'a que des associations modestes avec le revenu (0.34) et le profit (0.30), ce qui fait que le volume est un facteur secondaire par rapport à la tarification

Saving Pre-Processed Dataframe

In [50]:

```
df.to_csv('Sales_data(EDA Exported).csv', index=False)
```

Principaux Insights

Cycle Mensuel des Revenus : Les revenus sont restés stables autour de 26,5 M USD entre 2014 et 2017, sans pics saisonniers réguliers. La chute la plus marquée, autour de 21,2 M USD, s'est produite au début de 2017, ce qui suggère une perturbation ponctuelle possible.

Répartition par Canal :

Vente en gros : 54 %

Distributeurs : 31 %

Exportations : 15 % Cela présente une opportunité pour accroître la présence internationale.

Produits Principaux (Revenus) :

Produit 26 : 110 M USD

Produit 13 : 68–75 M USD

Les produits moins performants génèrent entre 52 et 57 M USD.

Marges Bénéficiaires : Les marges bénéficiaires varient entre 18 % et 60 %, sans corrélation claire avec le prix unitaire. Des bandes horizontales denses suggèrent des stratégies de tarification standardisées à travers les différentes gammes de produits.

Volume Saisonniers : Il n'y a pas de schéma mensuel fort, mais une légère augmentation des volumes est observée autour de mai-juin. La baisse de début 2017 ($\approx 21,2$ M USD) pourrait nécessiter une analyse approfondie.

Performance Régionale :

Californie : 230 M USD de revenus et 7 500+ commandes

Illinois/Floride/Texas : 85–110 M USD et 3 500–4 500 commandes

New York/Indiana : 54 M USD et 2 000+ commandes

.

Recommandations

Stratégie d'Outliers : Exclure ou formaliser les produits en gros et les SKUs promotionnels lors du calcul des moyennes pour éviter de fausser les résultats.

Amélioration des Marges : Appliquer les leviers de tarification des produits performants aux gammes intermédiaires et faibles, et réduire les coûts pour les produits sous-performants.

Croissance à l'Export : Investir dans des campagnes de marketing ciblées à l'international et renforcer les partenariats avec les distributeurs pour développer les exportations.

Planification Saisonnière : Réorienter les dépenses vers la période de baisse en janvier et de pic en mai-juin, tout en enquêtant sur l'anomalie de 2017.

Préparation du Dashboard : Construire des tables agrégées pour les séries temporelles, la répartition par canal et la performance des produits en vue de la création du tableau de bord Power BI.